

Implementation of Distributed Shor's Algorithm

Dinis Felgueiras, 102857

Professor Paulo Mateus Doctor Chrysoula Vlachou

2nd Semester 2023-2024

1st Cycle Integrated Project in Applied Mathematics and Computation

dinis.felgueiras@tecnico.ulisboa.pt

1 Introduction

The objective of this project is to implement the Distributed version of Shor's Algorithm suggested in the Research Paper "**Distributed Shor's Algorithm**" (by Ligang Xiao, Daowen Qiu, Le Luo and Paulo Mateus) utilizing Python, more specifically the package `Qiskit`.

2 Quantum Bits and Dirac Notation [1]

In Classical Computation we have the familiar concept to most of us, the bit. Analogously, in Quantum Computation and Quantum information we have the *Quantum Bit* or, as we shall mention throughout the rest of this report, the *qubit*.

In Classical Computation, a bit is a unit of data that has two possible states, 0 and 1. In Quantum Computation, a *qubit* is also a unit of data for which $|0\rangle$ and $|1\rangle$ are two possible states (the aforementioned notation is called Dirac Notation and it is used to denote states in quantum mechanics). Although $|0\rangle$ and $|1\rangle$ are two possible states, they are not the only two states available. In quantum mechanics, there is the concept of *superposition*, that is, a linear combination of states.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1)$$

While a bit is only in state 0 or 1, a *qubit* can exist in a continuum of states between 0 and 1, because of the ability of superposition.

Before moving on it is necessary to be familiarized a bit more with Dirac Notation. A *qubit* is represented as a vector in a two-dimensional Hilbert Space and it is written as $|\psi\rangle$. It's conjugate is represented as $\langle\psi|$. To represent the inner product between two vectors we shall use $\langle\psi|\phi\rangle$. The tensor product will be represented as $|\psi\rangle \otimes |\phi\rangle$, or simplified, $|\psi\rangle|\phi\rangle$.

Let A be a matrix. Its conjugate will be mentioned as A^* and it's transpose will be A^T and, finally, the hermitian conjugate matrix will be represented by $A^\dagger$:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad A^* = \begin{bmatrix} a^* & b^* \\ c^* & d^* \end{bmatrix} \quad (2)$$

$$A^T = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \quad A^\dagger = \begin{bmatrix} a^* & c^* \\ b^* & d^* \end{bmatrix} \quad (3)$$

The multiplication between a vector and a matrix will be represented as $A|\psi\rangle$ and the inner product between this vector and a vector ϕ will be written as $\langle\phi|A|\psi\rangle$.

There are a few matrices very important to Quantum Computation and are commonly referred to as the Pauli Matrices.

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad (4)$$

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad (5)$$

Besides the Pauli Matrices, there are a few more matrices important for building quantum circuits, the Hadamard gate (H), the phase gate (S) and the $\frac{\pi}{8}$ gate (T). These are represented by the following matrices:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \quad (6)$$

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \quad (7)$$

It is possible to derive three more important matrices from the Pauli Matrices aforementioned. When exponentiated, we obtain the rotation operators.

$$e^{-i\theta X/2} = R_x(\theta) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \quad (8)$$

$$e^{-i\theta Y/2} = R_y(\theta) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \quad (9)$$

$$e^{-i\theta Z/2} = R_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix} \quad (10)$$

3 Quantum Circuits [1] [2]

Just like in Classical Computing, Quantum Computing utilizes circuits as an abstract representation of logic operations. Circuits are usually implemented using digital logic gates. Essentially, circuits are composed of components that process input signals and produce output signals according to predefined rules.

Even though, in Classical Computing, there are only two universal gates (**NAND** and **NOR**), below we present some of the most common logical gates for classical computing:

- **NOT Gate:** Also known as an inverter, this gate has a single input and a single output. It simply negates the input, so if the input is 0, the output is 1, and vice versa.
- **AND Gate:** This gate has two or more inputs and a single output. If both inputs are high (1), the output is high (1). If any input is low (0), the output is low (0).
- **OR Gate:** Similar to the AND gate, the OR gate also has two or more inputs and a single output. The output is high if any of the inputs are high.
- **NAND Gate:** This gate is a combination of an AND gate followed by a NOT gate. The output of a NAND gate is low only when all inputs are high; otherwise, the output is high.
- **NOR Gate:** Similarly, the NOR gate is a combination of an OR gate followed by a NOT gate. The output of a NOR gate is high only when all inputs are low; otherwise, the output is low.
- **XOR Gate:** Exclusive OR gate has two inputs and one output. The output is high if the inputs are different; otherwise, the output is low.
- **XNOR Gate:** Exclusive NOR gate, also known as equivalence gate, produces a high output if both inputs are the same; otherwise, it produces a low output.

In Classical Computing, any logical circuit can be built utilizing only **NAND Gates**, and, consequently, the NAND gate is called universal¹.

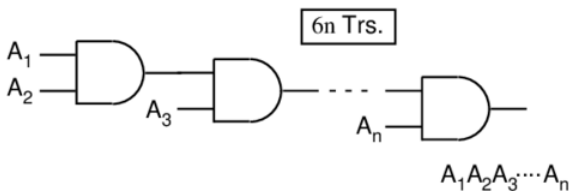


Fig. 1: Representation of an AND Gate

As we can see in Figure 1, there are multiple **AND Gates** applied to the several bits. The **AND Gate** takes two inputs and has only

one output, for example, in the first **AND Gate** applied the inputs are A_1 and A_2 and the output is A_1A_2 . The final output of the circuit is $A_1A_2...A_n$

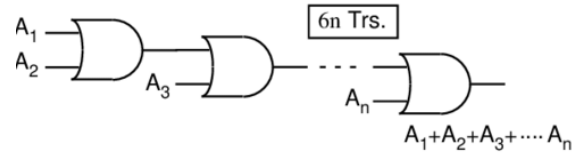


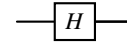
Fig. 2: Representation of an OR Gate

As we can see in Figure 2, there are multiple **OR Gates** applied to the several bits. The **OR Gate** takes two inputs and has only one output, for example, in the first **OR Gate** applied the inputs are A_1 and A_2 and the output is $A_1 + A_2$, where the $+$ represents or. The final output of the circuit is $A_1 + A_2 + ... + A_n$

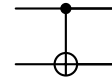
Now it is possible to discuss Quantum Circuits and Quantum Gates. As mentioned in the beginning of this section, Quantum Computing also utilizes circuits as an abstract representation of logic operations, those being represented with gates.

In Quantum Computing, it is necessary to consider the following set of four gates to have a universal gate set: **Hadamard Gate**, **Phase Gate**, **T Gate** and **CNOT Gate**. Below, these four gates are represented:

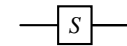
- **Hadamard Gate (H Gate):** Creates superposition on a qubit.



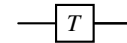
- **CNOT Gate (Controlled-NOT Gate):** Flips the target qubit conditionally based on the state of the control qubit.



- **Phase Gate (S Gate):** Introduces a phase shift of $\pi/2$ to the $|1\rangle$ state.

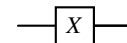


- **T Gate:** Introduces a phase shift of $\pi/4$ to the $|1\rangle$ state.



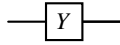
Below, additionally to these ones, there are a few more quantum gates presented.

- **Pauli-X Gate (X Gate):** Performs a bit-flip operation on a qubit.

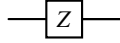


- **Pauli-Y Gate (Y Gate):** Performs a bit-flip operation and introduces a phase flip on a qubit.

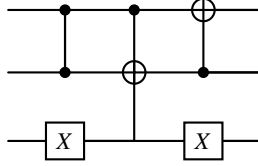
¹Note that NAND is not the only universal gate.



- **Pauli-Z Gate (Z Gate):** Flips the sign of the $|1\rangle$ state on a qubit.



- **Toffoli Gate (CCNOT Gate):** Flips the target qubit if both control qubits are in the $|1\rangle$ state.



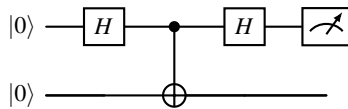
- **SWAP Gate:** Exchanges the state of two qubits.



Besides all these gates there is one more important representation, that is the **Measurement Gate**. The **Measurement Gate** represents a process that transitions a quantum state into a classical one.



Now that all the notation for some of the most relevant gates was seen, let's see an example on how a quantum circuit works.



In this circuit we initialize two qubits to the state $|0\rangle$. Next up, a **Hadamard Gate** is applied to first qubit, followed by a **CNOT Gate** and another **Hadamard Gate**. Finally, we use a **Measurement Gate** to measure the output of both qubits and record it as classical bits.

In summary, quantum circuits and their notation for quantum gates are fundamental elements of quantum computing. They provide a structured framework for designing and implementing quantum algorithms, allowing manipulation of quantum states to perform computations. Quantum gates, represented by symbols such as the **Pauli-X**, **Hadamard**, and **CNOT gates**, enable operations on qubits, facilitating tasks like creating superposition, entanglement, and executing quantum logic.

4 Quantum Algorithms [1][3]

The importance of Quantum Algorithms is all based on the ability to solve problems that, when resorting to Classical Computing require

much more resources and time. Quantum Algorithms can be separated into two main categories, the ones based on **Shor's Quantum Fourier Transform** (the one which this project will be based on) and the ones based on **Grover's Algorithm** for performing quantum search.

The **Quantum Fourier Transform** has applications within phase estimation, order-finding, factoring and discrete log problems.

On the other hand, **Grover's Algorithm**[3] has a lot of different applications, such as database search and counting and speedup of solutions of **NP-complete** problems.

In the picture below we can see the **Quantum Counting** as a combination of the two main algorithms.

In this project, the main focus is the **Quantum Fourier Transform**, as it is the base for **Shor's Algorithm**.

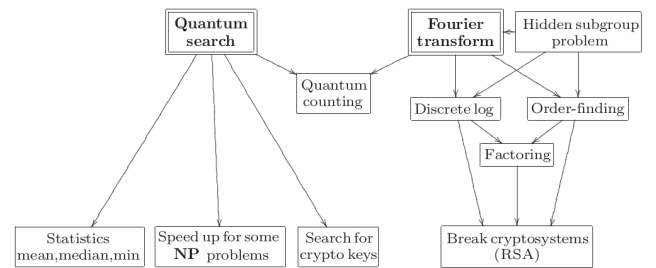


Fig. 3: Main Algorithms and Applications

5 Quantum Fourier Transform [1] [4] [5]

One of the biggest advantages of quantum computation arose when it was evident that some algorithms can be computed much faster on quantum computers than on classical computers. One of those algorithms is the **Discrete Fourier Transform (DFT)**. The QFT, the quantum algorithm for computing the DFT, has a lower time and space complexity than the FFT, the classical algorithm for computing DFT.

Discrete Fourier Transform: given a vector y of real or complex values applying the Discrete Fourier Transform to it gives us the following:

$$Y(k) = \sum_{n=0}^{N-1} y(n) e^{-2\pi i \frac{kn}{N}} \quad (11)$$

The output $Y(k)$ is a vector of the same length with output of complex numbers in each coordinate.

The **Quantum Fourier Transform** is the same transformation, but the notation used is slightly different. The **Quantum Fourier Transform** applied on an orthonormal basis $|0\rangle, \dots, |N-1\rangle$ is a linear operator that works like the following:

$$|j\rangle \longrightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k / N} |k\rangle \quad (12)$$

This transformation is unitary. There is also another way to represent the **Quantum Fourier Transform**, that is so useful that, sometimes, it is considered the definition of **Quantum Fourier Transform**.

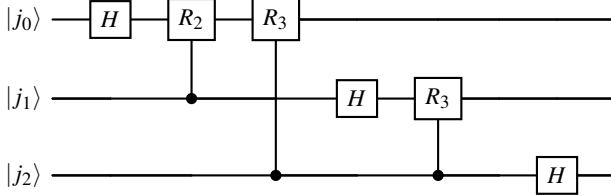
$$|j_1, \dots, j_n\rangle \rightarrow \frac{(|0\rangle + e^{2\pi i 0 \cdot j_n} |1\rangle) + (|0\rangle + e^{2\pi i 0 \cdot j_{n-1} j_n} |1\rangle) \dots}{2^{n/2}} \quad (13)$$

The formula above makes it easy to derive a efficient circuit for the **Quantum Fourier Transform**, that utilizes the gate R_k as the following unitary transformation:

$$R_k \equiv \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i / 2^k} \end{bmatrix} \quad (14)$$

5.1 Quantum Fourier Transform Circuit

To analyse the circuit we shall use as an example the following circuit:



As it is possible to observe, the circuit above applies a **Hadamard gate**, followed by 2 R gates to the first qubit. And it follows these steps for the other two qubits. To make sure this circuit performs the same as the formulas aforementioned, let us analyse the output of each qubit and compare it with the formulas.

Let j_0 be the first input. When applying the **Hadamard Gate** to it we obtain the following state:

$$\frac{1}{2^{1/2}} (|0\rangle + e^{2\pi i 0 \cdot j_0} |1\rangle) |j_1 j_2\rangle \quad (15)$$

Applying both R_k gates gives us the following state

$$\frac{1}{2^{1/2}} (|0\rangle + e^{2\pi i 0 \cdot j_0 j_1} |1\rangle) |j_1 j_2\rangle \quad (16)$$

Now, a similar process for the second qubit. By applying the **Hadamard Gate** we obtain the following state:

$$\frac{1}{2^{2/2}} (|0\rangle + e^{2\pi i 0 \cdot j_0 j_1} |1\rangle) |j_1 j_2\rangle (|0\rangle + e^{2\pi i 0 \cdot j_1} |1\rangle) |j_2\rangle \quad (17)$$

Finally applying the same process to third qubit we obtain:

$$\frac{1}{2^{3/2}} (|0\rangle + e^{2\pi i 0 \cdot j_0 j_1 j_2} |1\rangle) (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2} |1\rangle) (|0\rangle + e^{2\pi i 0 \cdot j_2} |1\rangle) \quad (18)$$

Applying the formula in **Equation 13** with $n = 3$ we obtain:

$$\frac{1}{2^{3/2}} (|0\rangle + e^{2\pi i 0 \cdot j_0 j_1 j_2} |1\rangle) (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2} |1\rangle) (|0\rangle + e^{2\pi i 0 \cdot j_2} |1\rangle) \quad (19)$$

This allows us to conclude that the circuit is the same as applying the formula.

Now let us analyse the complexity of the algorithm. The first step of the algorithm is applying a **Hadamard gate** and $n - 1$ conditional rotations on the first qubit. Then a similar process the second qubit, with the minus 1 conditional rotation. This gives us the following:

$$\sum_{i=0}^{n-1} i + 1 = n(n+1)/2 \quad (20)$$

But this sum doesn't include the potential swaps, that are, at most, $n/2$. Each swap involves 3 **controlled-NOT gates**. So this gives us a complexity of $\mathcal{O}(n^2)$ for performing the **Quantum Fourier Transform**. This complexity is exponentially better than the best classical algorithms for computing the **Discrete Fourier Transform**. This complexity is also the complexity for the **Inverse Quantum Fourier Transform**, that will be used in the implementation of **Shor's Algorithm**.

This also allows to conclude that the **Quantum Fourier Transform** is unitary, since all the gates utilized in the circuit are unitary.

5.2 Quantum Fourier Transform Algorithm

Below, the Python code for the **Quantum Fourier Transform**, that follows the steps mentioned earlier is presented. First we apply **Hadamard Gates**, followed by **Controlled-Phase Gates**.

```
1 from qiskit import QuantumCircuit, transpile
2 from qiskit_visualization import circuit_drawer
3 import numpy as np
4
5 def qft_circuit(n):
6     circuit = QuantumCircuit(n)
7
8     # Apply Hadamard gates and controlled-phase gates
9     for i in range(n):
10         circuit.h(i)
11         for j in range(i+1, n):
12             circuit.cp(np.pi/(2**(j-i)), j, i)
13
14     return circuit
15
16 n_qubits = 3 # Specify the number of qubits
17 qft_circuit = qft_circuit(n_qubits)
18
19 transpiled_circuit = transpile(qft_circuit,
20                                optimization_level=0) # Transpile for better
21                                visualization
22 circuit_drawer(transpiled_circuit, output='mpl', scale=0.75)
```

Listing 1: Quantum Fourier Transform Circuit Function

Besides doing the calculations, this code also prints the resulting circuit, to make sure it follows the same rules as the theoretical circuit.



For the implementation of Shor's Algorithm, the built-in function `QuantumFourierTransform` shall be used, as a way to simplify the code. Even though the built-in function will be used, it is very important to understand the steps of the **Quantum Fourier Transform**, as it is the foundation for many quantum algorithms.

One of the many important applications of the **Quantum Fourier Transform** is the **Phase Estimation Algorithm**. Let us assume U an unitary operator, $|u\rangle$ as it's eigenvector and the corresponding eigenvalue $e^{2\pi i\phi}$, where ϕ is an unknown constant. The **Phase Estimation Algorithm** is able to estimate ϕ . To perform this algorithm it is necessary to have oracles available. The oracles are responsible for performing the modular exponentiation.

The **Phase Estimation Algorithm** is an essential subroutine of the **Order Finding Algorithm**.

Below, there is the implementation of the **Phase Estimation Algorithm**. This Algorithm follows these next steps:

Algorithm 1 Phase Estimation Algorithm

```

1: procedure
2:   Create initial state  $|0\rangle^{\otimes t} |u\rangle$ 
3:   Apply  $H^{\otimes t}$  to the first register
4:   Apply  $C_t(U)$ 
5:   Apply  $QFT^{-1}$ 
6:   Measure the first register
7: end procedure

```

First, we create the initial state, followed by applying **Hadamard Gates** to all the qubits in the first register. Next, we apply the unitary transformation (when applying the **Order Finding Algorithm** instead, the unitary transformation is the controlled modular exponentiation), followed by the **Inverse Quantum Fourier Transform** and measurement of the first register.

```
50 print("Approximated phase:", estimated_phase)
```

Listing 2: Phase Estimation Algorithm Function

5.4 Order Finding

The purpose of **Order Finding Algorithm** is to find r of a modulo N , i.e, the smallest r such that $a^r \equiv 1 \pmod{N}$. For this algorithm we define the following unitary operator:

$$M_a |x\rangle = |ax \bmod N\rangle \quad (21)$$

This step preforms the Modular Exponentiation, the first step of **Shor's Algorithm**. This step is responsible of turning our factorization problem into a period finding problem. Afterwards, it is necessary to apply the **Inverse Quantum Fourier Transform**, to obtain the period of it.

5.4.1 Order Finding Algorithm

The following Pseudo-Code applies the **Order-Finding Algorithm** as suggested in the paper "*Distributed Shor's Algorithm*", by Ligang Xiao, Daowen Qiu, Le Luo and Paulo Mateus.

Algorithm 2 Order Finding Algorithm

```
1: procedure
2:   Create initial state  $|0\rangle^{\otimes t} |1\rangle$ 
3:   Apply  $H^{\otimes t}$  to the first register
4:   Apply  $C_t(M_a)$ 
5:   Apply  $QFT^{-1}$ 
6:   Measure the first register
7:   Apply continued fractions algorithm
8: end procedure
```

5.4.2 Continued Fractions Method and Classical Post-Processing

In **Shor's Algorithm**, the continued fractions algorithm is an essential step in the classical post-processing, after the quantum part has been executed. Specifically, it is used to find the approximate period of a modular exponentiation function, which is essential for factorizing composite integers.

A simple continued fraction can be represented as:

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots + \frac{1}{a_n}}}}$$

By iteratively generating convergents of the period, the continued fractions algorithm allows for the rapid identification of rational approximations that guide the determination of potential factors, contributing to the overall efficiency and success of Shor's algorithm in integer factorization.

After obtaining the period through **Continued Fractions Algorithm** it is necessary to finish the algorithm.

5.4.3 Continued Fractions Method and Classical Post-Processing Algorithms

The code below is the application of the **Continued Fractions Algorithm** and the **Post-Processing** as a way to finalize **Shor's Algorithm**.

Algorithm 3 Post-Processing

```
1: procedure
2:   If  $r$  is odd go back to quantum part, if  $r$  is even advance
3:   Define  $x$  as  $a^{\frac{r}{2}}$ 
4:   Find  $GCD(x+1, N)$  and  $GCD(x-1, N)$ 
5:   If  $p \neq 1$  and  $p \neq N$  return  $(p, N//p)$ 
6:   If not, check same for  $q$  and return  $(q, N//q)$ 
7:   If not, return None
8: end procedure
```

5.4.4 Continued Fractions Method and Classical Post-Processing Implementation

```
1 def continued_fractions_algorithm(x, epsilon=1e-6):
2     fractions = []
3     while abs(x) >= epsilon:
4         integer_part = math.floor(x)
5         fractions.append(integer_part)
6         if abs(x - integer_part) < epsilon:
7             break # Break the loop if the fractional
8                 # part is within epsilon
9         x = 1 / (x - integer_part)
10    return fractions
11
12 def best_r_from_continued_fractions(fractions):
13     n0, n1 = fractions[0], fractions[1]
14     for i in range(2, len(fractions)):
15         n2 = fractions[i] * n1 + n0
16         n0, n1 = n1, n2
17     return n1
18
19 def post_process_factors(r, a, N):
20     # Check if r is odd
21     if r % 2 != 0:
22         return None # r is odd, cannot continue
```



```

23
24     # Calculate x
25     x = pow(a, r // 2, N)
26
27     # Find factors
28     p = gcd(x + 1, N)
29     q = gcd(x - 1, N)
30
31     if p != 1 and p != N:
32         return (p, N // p)
33     elif q != 1 and q != N:
34         return (q, N // q)
35     else:
36         return None # Factor not found, try a
                        different 'a'

```

Listing 3: Continued Fractions Algorithm and Post-Processing

5.4.5 Order Finding Algorithm Implementation

The following implementation is the implementation of the Pseudo-Code in Algorithm 2.

The inputs for this algorithm are the number of qubits, the base of the modular exponentiation, the modulus for the exponentiation and the initial state of the circuit.

This code returns the factors (if found), draws the quantum circuit utilized for running the code and will give us the distribution of the qubits measured.

```

1 from qiskit import QuantumCircuit, transpile
2 from qiskit_aer import Aer
3 import numpy as np
4 import math
5 from qiskit.circuit.library import QFT
6 from qiskit.circuit.library.standard_gates import
   U1Gate
7 from qiskit import ClassicalRegister
8 import matplotlib.pyplot as plt
9 from math import gcd
10 from fractions import Fraction
11
12 # Define the modular exponentiation gate
13 def controlled_modular_exponentiation(qc, control,
   target, a, exponent, modulus):
14     qc.append(U1Gate((2 * np.pi * a**exponent) /
   modulus).control(), [control] + [target])
15
16 # Define the function to create the initial state
17 def create_initial_state(L):
18     # Calculate the value of t
19     t = 2 * L + 1 + math.ceil(math.log2(2))
20
21     # Create a quantum circuit with t+L qubits
22     qc = QuantumCircuit(t + L, L)
23

```

```

24     # Apply X gate to flip the second register to |1
   state
25     qc.x(range(t, t + L))
26
27     # Apply Hadamard gate to all qubits in the first
   register
28     qc.h(range(t))
29
30     return qc
31
32 # Apply controlled modular exponentiation gate to every
   qubit in the first register
33 def apply_controlled_modular_exponentiation_to_all(qc,
   t, a, modulus):
34     for qubit in range(t):
35         controlled_modular_exponentiation(qc, qubit +
   1, 0, a, qubit, modulus)
36
37 def continued_fractions_algorithm(x, epsilon=1e-6):
38     fractions = []
39     while abs(x) >= epsilon:
40         integer_part = math.floor(x)
41         fractions.append(integer_part)
42         if abs(x - integer_part) < epsilon:
43             break # Break the loop if the fractional
   part is within epsilon
44         x = 1 / (x - integer_part)
45     return fractions
46
47 def best_r_from_continued_fractions(fractions):
48     n0, n1 = fractions[0], fractions[1]
49     for i in range(2, len(fractions)):
50         n2 = fractions[i] * n1 + n0
51         n0, n1 = n1, n2
52     return n1
53
54 def post_process_factors(r, a, N):
55
56     # Check if r is odd
57     if r % 2 != 0:
58         return None # r is odd, cannot continue
59
60     # Calculate x
61     x = pow(a, r // 2, N)
62
63     # Find factors
64     p = gcd(x + 1, N)
65     q = gcd(x - 1, N)
66
67     if p != 1 and p != N:
68         return (p, N // p)
69     elif q != 1 and q != N:
70         return (q, N // q)
71     else:
72         return None # Factor not found, try a

```

```

different 'a'
73
74
75 # Example usage
76 L = 3
77 initial_state_circuit = create_initial_state(L)
78
79 # Apply controlled modular exponentiation gate to every
    qubit in the first register
80 a = 2 # Choose the base 'a' of the modular
    exponentiation
81 modulus = 15 # The modulus for the modular
    exponentiation
82 apply_controlled_modular_exponentiation_to_all(
    initial_state_circuit, L, a, modulus)
83
84 # Apply Inverse QFT
85 qft_dagger = QFT(num_qubits=L, inverse=True)
86 initial_state_circuit.append(qft_dagger, range(L))
87
88 # Measure the first register
89 initial_state_circuit.measure(range(L), range(L))
90
91 # Transpile the circuit
92 backend = Aer.get_backend('qasm_simulator')
93 transpiled_circuit = transpile(initial_state_circuit,
    backend=backend)
94
95 # Print the transpiled circuit
96 print(transpiled_circuit.draw())
97
98 # Simulate the transpiled circuit
99 job = backend.run(transpiled_circuit)
100 result = job.result()
101 counts = result.get_counts()
102
103 # Extract measurement outcomes and corresponding
    frequencies
104 measurement_outcomes = list(counts.keys())
105 frequencies = list(counts.values())
106
107 # Plot the counts
108 plt.figure(figsize=(8, 6))
109 plt.bar(measurement_outcomes, frequencies, color='b')
110 plt.xlabel('Measurement Outcome')
111 plt.ylabel('Frequency')
112 plt.title('Measurement Outcomes')
113 plt.xticks(rotation=45)
114 plt.show()
115
116 # Extract measurement result
117 m_decimal = int(list(counts.keys())[0], 2)
118 m = m_decimal / (2**(L))
119 print(m)
120

```

```

121 # Apply continued fractions algorithm to obtain r
122 fractions = continued_fractions_algorithm(m)
123 r = best_r_from_continued_fractions(fractions)
124 print("Fractions:", fractions)
125 print("Estimated r:", r)
126
127 # Post-process to find factors
128 factors = post_process_factors(r, a, modulus)
129 print("Factors:", factors)

```

Listing 4: Order Finding Algorithm Function

The outputs for $N=21$ and $a=2$, for a successful simulation, are presented in Figures 5 and 6.

Figures 5 and 6 present the quantum circuit that the code utilizes.

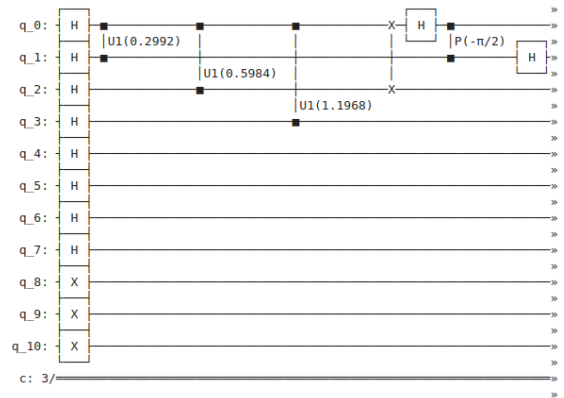


Fig. 5: First Part of the Drawn Quantum Circuit

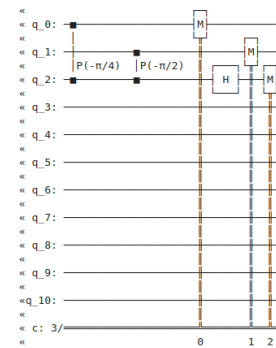


Fig. 6: Second Part of the Drawn Quantum Circuit

The drawing of the circuit makes this code much less efficient (this algorithm is only capable of running in reasonable times for very small values of N), so in the next chapter we will present a different implementation.

6 Shor's Algorithm [4] [6] [7] [5]

As mentioned previously, **Shor's Algorithm** can be divided into two stages: the **Modular Exponentiation** and the **Inverse Quantum Fourier Transform**. The **Modular Exponentiation** transforms the

factorization problem into a period finding problem. These types of problems can be efficiently solved by the **Inverse Quantum Fourier Transform**.

As it was mentioned above, the code provided earlier does not compute factors for large values of N efficiently, because of the circuit drawing and the table with measurement outcomes. The code below is able to perform **Shor's Algorithm** for much larger values of N . Besides that, the code is also written such that it performs the algorithm 1000 times or until it finds a factor (whichever happens first).

This stopping method is very important, because of the similarities between **Shor's Algorithm** and a Las Vegas Algorithm. Like a Las Vegas Algorithm, it is necessary to run **Shor's Algorithm** several times until the correct response is obtained.

6.1 Shor's Algorithm Implementation

```
1 from qiskit import QuantumCircuit, transpile
2 from qiskit_aer import Aer
3 import numpy as np
4 import math
5 from qiskit.circuit.library import QFT
6 from qiskit.circuit.library.standard_gates import
   U1Gate
7 from qiskit import ClassicalRegister
8 import matplotlib.pyplot as plt
9 from math import gcd
10 from fractions import Fraction
11 import time
12
13 # Define the modular exponentiation gate
14 def controlled_modular_exponentiation(qc, control,
   target, a, exponent, modulus):
15     qc.append(U1Gate((2 * np.pi * a**exponent) /
   modulus).control(), [control] + [target])
16
17 # Define the function to create the initial state
18 def create_initial_state(L):
19     # Calculate the value of t
20     t = 2 * L + 1 + math.ceil(math.log2(2))
21
22     # Create a quantum circuit with t+L qubits
23     qc = QuantumCircuit(t + L, L)
24
25     # Apply X gate to flip the second register to |1
   state
26     qc.x(range(t, t + L))
27
28     # Apply Hadamard gate to all qubits in the first
   register
29     qc.h(range(t))
30
31     return qc
```

```
32
33 # Apply controlled modular exponentiation gate to every
   qubit in the first register
34 def apply_controlled_modular_exponentiation_to_all(qc,
   t, a, modulus):
35     for qubit in range(t):
36         controlled_modular_exponentiation(qc, qubit +
   1, 0, a, qubit, modulus)
37
38 def continued_fractions_algorithm(x, epsilon=1e-6):
39     fractions = []
40     while abs(x) >= epsilon:
41         integer_part = math.floor(x)
42         fractions.append(integer_part)
43         if abs(x - integer_part) < epsilon:
44             break # Break the loop if the fractional
   part is within epsilon
45         x = 1 / (x - integer_part)
46     return fractions
47
48 def best_r_from_continued_fractions(fractions):
49     n0, n1 = fractions[0], fractions[1]
50     for i in range(2, len(fractions)):
51         n2 = fractions[i] * n1 + n0
52         n0, n1 = n1, n2
53     return n1
54
55 def post_process_factors(r, a, N):
56
57     # Check if r is odd
58     if r % 2 != 0:
59         return None # r is odd, cannot continue
60
61     # Calculate x
62     x = pow(a, r // 2, N)
63
64     # Find factors
65     p = gcd(x + 1, N)
66     q = gcd(x - 1, N)
67
68     if p != 1 and p != N:
69         return (p, N // p)
70     elif q != 1 and q != N:
71         return (q, N // q)
72     else:
73         return None # Factor not found, try a
   different 'a'
74
75 # Main function to run the factoring algorithm
76 def run_shor_algorithm(L, a, modulus, max_iterations
   =1000):
77     backend = Aer.get_backend('qasm_simulator')
78     factors_found = False
79     iterations = 0
80
```

```

81 start_time = time.time()
82 while not factors_found and iterations <
max_iterations:
83     iterations += 1
84     print("Iteration:", iterations)
85
86     # Create initial state circuit
87     initial_state_circuit = create_initial_state(L)
88
89     # Apply controlled modular exponentiation gate
to every qubit in the first register
90     apply_controlled_modular_exponentiation_to_all(
initial_state_circuit, L, a, modulus)
91
92     # Apply Inverse QFT
93     qft_dagger = QFT(num_qubits=L, inverse=True)
94     initial_state_circuit.append(qft_dagger, range(
L))
95
96     # Measure the first register
97     initial_state_circuit.measure(range(L), range(L
))
98
99     # Transpile the circuit
100    transpiled_circuit = transpile(
initial_state_circuit, backend=backend)
101
102    # Simulate the transpiled circuit
103    job = backend.run(transpiled_circuit)
104    result = job.result()
105    counts = result.get_counts()
106
107    # Extract measurement outcome
108    measurement_outcomes = list(counts.keys())
109    m_decimal = int(measurement_outcomes[0], 2)
110    m = m_decimal / (2**(L))
111
112    # Apply continued fractions algorithm to obtain
r
113    fractions = continued_fractions_algorithm(m)
114    r = best_r_from_continued_fractions(fractions)
115    print("Fractions:", fractions)
116    print("Estimated r:", r)
117
118    # Post-process to find factors
119    factors = post_process_factors(r, a, modulus)
120    print("Factors:", factors)
121
122    if factors is not None:
123        factors_found = True
124        print("Factors found:", factors)
125    else:
126        print("Factors not found, trying again...")
127
128    end_time = time.time()

```

```

129 elapsed_time = end_time - start_time
130 print("Elapsed Time:", elapsed_time, "seconds")
131
132 if not factors_found:
133     print("Max iterations reached. Factors not
found.")
134
135 # Example usage
136 L = 9
137 a = 2 # Choose the base 'a' of the modular
exponentiation
138 modulus = 1927 # The modulus for the modular
exponentiation
139
140 run_shor_algorithm(L, a, modulus)

```

Listing 5: Phase Estimation Algorithm Function

To demonstrate the use of this algorithm in factoring numbers that are the product of two primes we have the following table, that shows the N to be factored, the average number of iterations over 10 tries, the factors and the average time it took and the average deviation.

Table 1: Results of Shor's Algorithm for several values of N

N	Average No. of Iterations	Factors Found	Average Time	Deviation
15	3.5	(5,3)	0.201 sec	0.229 sec
21	5.8	(3,7)	0.152 sec	0.152 sec
35	5.2	(5,7)	0.134 sec	0.102 sec
143	20.2	(11,13)	0.849 sec	0.609 sec
247	9.6	(19,13)	0.678 sec	0.281 sec
713	21.3	(31,23)	1.066 sec	0.434 sec
1927	236.2	(47,41)	6.554 sec	4.223 sec
3127	296.7	(59,53)	8.319 sec	7.194 sec
4757	423.6	(67,71)	10.753 sec	4.929 sec
6059	54.9	(73,93)	1.738 sec	1.399 sec

7 Quantum Entanglement

To represent the information of two bits simultaneously, the tensor product is used, that works as follows:

$$\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix} \quad (22)$$

In the case two qubits are not *entangled*, from the final state it is possible to obtain the initial qubits.

$$\begin{pmatrix} ac = 1 \\ ad = 0 \\ bc = 0 \\ bd = 0 \end{pmatrix} = \begin{pmatrix} a = 1 \\ b = 0 \end{pmatrix} \otimes \begin{pmatrix} c = 1 \\ d = 0 \end{pmatrix} \quad (23)$$

So, according to the example above, the qubits are not entangled, since the initial states are easily obtainable from the final state.

Two qubits are said to be *entangled* if it is not possible to factorize the product between the two.

But, in the following case the qubits are entangled, since it is impossible to get the initial states back.

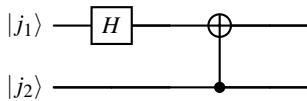
$$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix} \rightarrow \begin{matrix} ac = \frac{1}{\sqrt{2}} \\ ad = 0 \\ bc = 0 \\ bd = \frac{1}{\sqrt{2}} \end{matrix} \quad (24)$$

The equations above have no solution, so our two qubits are *entangled*.

By two qubits being *entangled*, the two share information, no matter the distance between them.

This is an essential concept for **Quantum Teleportation**, that is going to be explained right ahead.

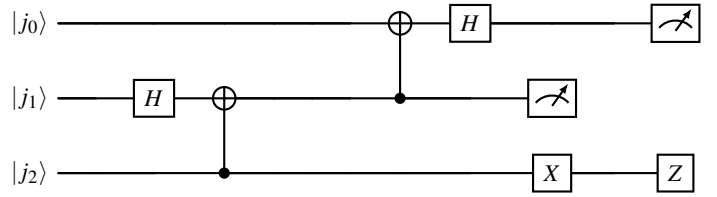
The entanglement between two qubits in quantum circuits is represented as:



8 Quantum Teleportation [1] [4] [8]

A fundamental part of the **Distributed Shor's Algorithm** is **Quantum Teleportation**. **Quantum Teleportation** is basically equivalent to physically teleporting the qubits. This concept can also be explained more clearly in terms of two parties, Bob and Alice.

Let us assume that Alice wants to teleport her qubit to Bob, that is, wants to transport her qubit to Bob through time and space. That is only possible resorting to **Quantum Entanglement**. This concept is much easier to understand and explain utilizing a Quantum Circuit.



We have a three qubit system, to help us understand **Quantum Teleportation** of qubit $|j_0\rangle$, that belongs to Alice. Qubit $|j_1\rangle$ is also Alice's and $|j_2\rangle$ belongs to Bob. The first step is applying a Hadamard Gate to qubit $|j_1\rangle$, to initialize it, and entangle it with Bob's qubit ($|j_2\rangle$). Next, Alice *entangles* her two qubits and applies a Hadamard Gate to qubit $|j_0\rangle$. Finally, Alice measures her two qubits, which results two classical bits. Depending on the output of the measurements, Bob will be able to decode $|j_1\rangle$ by applying the correct unitary transformation.

From this, there is a very important **theorem** to note.

Theorem: When Alice and Bob share L pairs of EPR pairs, they can simulate transmitting L qubits by communicating $2L$ classical bits

So, **Quantum Teleportation** is an essential concept in Distributed Quantum Algorithms, since it allows us to have multiple computers communicate through this process.

9 Distributed Order Finding [4]

Shor's Algorithm was first proposed in 1997, by Peter Shor and, since, many distributed versions were suggested, as a way to optimize this essential algorithm in quantum computation.

In the paper **"Distributed Shor's Algorithm"**, by Ligang Xiao, Daowen Qiu, Le Luo and Paulo Mateus, a Distributed version of the **Order Finding Algorithm** was suggested.

To implement this version it would be necessary to have two quantum computers (Computer A and Computer B) that communicate via **Quantum Teleportation**.

Let us assume we are working with $\frac{L}{2} + 1$ qubits with computer A and $\frac{3L}{2} + 2$ qubits with computer B, where L is a even number.

First, we apply the **Order Finding Algorithm** in Computer A. That gives us an estimation of $\frac{s}{r}$ the first $\frac{L}{2} + 1$ qubits.

Then, we obtain the same estimation from computer B, for qubits between $\frac{L}{2} + 2$ and $2L + 1$. This is possible by applying a slightly different way of the modular exponentiation. Instead of just applying $C_t(M_a)$, it is necessary to apply $C_t(M_a^{2^t})$.

In order to verify that Computer A and B measure the same $\frac{s}{r}$ we need to apply a **Correction Algorithm**, that is discussed further ahead.

Even though it is only essential for Computer B to estimate from qubit $\frac{L}{2} + 2$ to $2L + 1$, in order to increase the precision of the algo-

rithm, Computer B also estimates $\frac{L}{2} + 1$ and compares it with the one calculated in Computer A.

Below there is an image that represents the circuit correspondent to the **Distributed Version of Shor's Algorithm**.

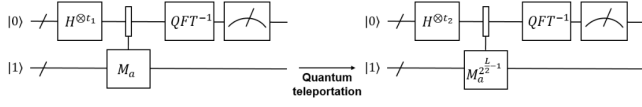


Fig. 7: Distributed Version of Shor's Algorithm

As it is evident from the image, both the computers perform the same operations, the **Modular Exponentiation** and the **Inverse Quantum Fourier Transform**. After measuring the results of Computer A, **Quantum Teleportation** is used, to take the information from A to B. Afterwards, there will be a subroutine that needs to be applied, followed again by the **Modular Exponentiation** (now every exponentiation is squared, to perform the second part of the **Modular Exponentiation**) and the **Inverse Quantum Fourier Transform**.

Finally, the same Classical Post-Processing is applied, to obtain the factors of the number.

9.1 Distributed Order Finding Algorithm

Below, is the pseudo-code suggested in [4].

Algorithm 4 Distributed Order Finding Algorithm

```

1: procedure
2:   Computer A creates the initial state  $|0\rangle_A |1\rangle_C$  and Computer
   B creates the initial state  $|0\rangle_B$ 
3:   Apply  $H^{\otimes t_1}$  to the register A
4:   Apply  $C_{t_1}(M_a)$  to registers A and C
5:   Apply  $QFT^{-1}$  to register A
6:   Teleport the qubits of register C to register B
7:   Apply  $H^{\otimes t_2}$  to the register B
8:   Apply  $C_{t_2}((M_a^2)^{\frac{L}{2}-1})$  to registers A and C
9:   Apply  $QFT^{-1}$  to register B
10:  Computer A measures register A and Computer B measures
    register B
11:  Apply Correct Results Algorithm
12:  Apply Continued Fractions Algorithm
13: end procedure

```

9.2 Correctness of the Algorithm

To ensure the correctness of the algorithm it is necessary to apply a subroutine.

In the distributed version of quantum algorithms, a correct results algorithm is essential to ensure that the distributed computation is accurate and reliable, despite the susceptibility to errors, due to quantum noise. This subroutine plays a crucial role in managing error correction, coordinating results from different computers to maintain consistency and optimizing resource utilization.

By addressing these factors, a correct results algorithm ensures that the distributed quantum system operates efficiently and produces valid results, which is vital for scaling up quantum computations and leveraging their full potential for complex problem-solving.

9.2.1 Correct Results Algorithm

This Algorithm is important for **Distributed Shor's Algorithm** if it is being implemented between two different quantum computers. In the implementation presented here, this subroutine will not be necessary, since we assume the result is correct, because teleportation is all being made in the same circuit.

Algorithm 5 Correct Results Algorithm

```

1: procedure
2:   Choose CorrectionBit
3:   Apply  $H^{\otimes t_1}$  to the register A
4:    $m_{prefix} \leftarrow (IBits(m_1, 1, \frac{L}{2} + 1) + CorrectionBit)$ 
5:    $m \leftarrow m_{prefix} \odot IBits(m_2, 3, t_2)$ 
6: end procedure

```

9.3 Quantum Teleportation Implementation

To apply **Quantum Teleportation** using Qiskit we shall measure everything in register A, then swap to register B and then erase everything in register A. This is the most efficient way to implement **Quantum Teleportation**, as teleporting the information is just replicating it and erasing everything that was at the source.

Therefore, to apply **Quantum Teleportation** it was necessary to define some new functions, such as one that applies the controlled modular exponentiation only to half of the register at a time, a function to swap the first half with the second and a function that would erase the information of the first half.

```

1 # Function to swap the first half and the second half
  of qubits
2 def swap_first_and_second_half(qc, t):
3     for i in range(L//2):
4         qc.swap(i, i + L//2)
5
6 # Function to reset the first half of qubits to |0
  state
7 def reset_first_half(qc, t):
8     for i in range(t//2):
9         qc.reset(i)
10

```

```

11 def apply_controlled_modular_exponentiation_to_A(qc, t,
12     a, modulus):
13     for qubit in range(t//2):
14         controlled_modular_exponentiation(qc, qubit, t,
15             a, qubit, modulus)
16
17 def apply_controlled_modular_exponentiation_to_B(qc, t,
18     a, modulus):
19     for qubit in range(t//2, t):
20         controlled_modular_exponentiation(qc, qubit, t,
21             a, qubit, modulus)

```

Listing 6: Phase Estimation Algorithm Function

These four functions are the ones used to implement the **Quantum Teleportation**.

The `swap_first_and_second_half` function is responsible of sending the information in the first half of qubits to the second half.

The `reset_first_half` function is responsible of deleting all of the information of the first half of qubits, to abide to the **No-Cloning Theorem**.

The `apply_controlled_modular_exponentiation_to_A` is responsible for applying the modular exponentiation to first half of the qubits.

Similarly `apply_controlled_modular_exponentiation_to_B`, is responsible for applying to the second half.

10 Distributed Shor's Algorithm [4]

In the **Distributed Shor's Algorithm** two computers work together utilizing **Quantum Teleportation**. This is a particularly useful result, since it allows us to reduce the number of qubits utilized by each computer and, as we will see further ahead, it allows us to reduce the circuit depth.

The ability to implement **Quantum Algorithms** using distributed computing unleashes a new era of computational power. By tapping into the parallel processing abilities of interconnected quantum computers across various locations, it promises exponential speedups for specific problems.

10.1 Distributed Shor's Algorithm Implementation

```

1 from qiskit import QuantumCircuit, transpile
2 from qiskit_aer import Aer
3 import numpy as np
4 import math
5 from qiskit.circuit.library import QFT
6 from qiskit.circuit.library.standard_gates import
7     U1Gate
8 from qiskit import ClassicalRegister
9 import matplotlib.pyplot as plt
10 from math import gcd

```

```

10 from fractions import Fraction
11 import time
12
13 # Define the modular exponentiation gate
14 def controlled_modular_exponentiation(qc, control,
15     target, a, exponent, modulus):
16     qc.append(U1Gate((2 * np.pi * a**exponent) /
17         modulus).control(), [control, target])
18
19 # Define the function to create the initial state
20 def create_initial_state(L):
21     # Calculate the value of t
22     t = 2 * L + 2
23
24     # Create a quantum circuit with t+L qubits and L
25     # classical bits for measurements
26     qc = QuantumCircuit(t + L, t)
27
28     # Apply X gate to flip the second register to |1
29     # state
30     qc.x(range(t, t + L))
31
32     # Apply Hadamard gate to all qubits in the first
33     # register
34     qc.h(range(t))
35
36     return qc
37
38 # Function to swap the first half and the second half
39 # of qubits
40 def swap_first_and_second_half(qc, t):
41     for i in range(L//2):
42         qc.swap(i, i + L//2)
43
44 # Function to reset the first half of qubits to |0
45 # state
46 def reset_first_half(qc, t):
47     for i in range(t//2):
48         qc.reset(i)
49
50 def
51     apply_controlled_modular_exponentiation_to_computerA
52     (qc, t, a, modulus):
53     for qubit in range(t//2):
54         controlled_modular_exponentiation(qc, qubit, t,
55             a, qubit, modulus)
56
57 def
58     apply_controlled_modular_exponentiation_to_computerB
59     (qc, t, a, modulus):
60     for qubit in range(t//2, t):
61         controlled_modular_exponentiation(qc, qubit, t,
62             a, qubit, modulus)

```

```

52 def continued_fractions_algorithm(x, epsilon=1e-6):
53     fractions = []
54     while abs(x) >= epsilon:
55         integer_part = math.floor(x)
56         fractions.append(integer_part)
57         if abs(x - integer_part) < epsilon:
58             break # Break the loop if the fractional
                    part is within epsilon
59         x = 1 / (x - integer_part)
60     return fractions
61
62 def best_r_from_continued_fractions(fractions):
63     n0, n1 = fractions[0], fractions[1]
64     for i in range(2, len(fractions)):
65         n2 = fractions[i] * n1 + n0
66         n0, n1 = n1, n2
67     return n1
68
69 def post_process_factors(r, a, N):
70
71     # Check if r is odd
72     if r % 2 != 0:
73         return None # r is odd, cannot continue
74
75     # Calculate x
76     x = pow(a, r // 2, N)
77
78     # Find factors
79     p = gcd(x + 1, N)
80     q = gcd(x - 1, N)
81
82     if p != 1 and p != N:
83         return (p, N // p)
84     elif q != 1 and q != N:
85         return (q, N // q)
86     else:
87         return None # Factor not found, try a
                    different 'a'
88
89 # Main function to run the factoring algorithm
90 def run_shor_algorithm(L, a, modulus, max_iterations
                        =1000):
91     backend = Aer.get_backend('qasm_simulator')
92     factors_found = False
93     iterations = 0
94
95     start_time = time.time()
96     while not factors_found and iterations <
        max_iterations:
97         iterations += 1
98         print("Iteration:", iterations)
99
100         # Create initial state circuit
101         initial_state_circuit = create_initial_state(L)

```

```

103         # Calculate the value of t
104         t = 2 * L + 2
105
106         # Apply controlled modular exponentiation gate
                    to every qubit in the first half of the register
107
108         apply_controlled_modular_exponentiation_to_computerA
                    (initial_state_circuit, L, a, modulus)
109
110         # Apply Inverse QFT to the first half of the
                    qubits
111         qft_dagger = QFT(num_qubits=L//2, inverse=True)
112         initial_state_circuit.append(qft_dagger, range(
                    L//2))
113
114         # Measure the first half of the register
115         initial_state_circuit.measure(range(L//2),
                    range(L//2))
116
117         # Swap the first half and the second half of
                    qubits
118         swap_first_and_second_half(
                    initial_state_circuit, L//2)
119
120         # Measure the time for the first segment
121         mid_time = time.time()
122         first_segment_time = mid_time - start_time
123         print("Time for first segment:",
                    first_segment_time, "seconds")
124
125         # Reset the first half of the qubits to |0
                    state
126         reset_first_half(initial_state_circuit, L//2)
127
128         # Apply controlled modular exponentiation gate
                    to every qubit in the second half of the register
129
130         apply_controlled_modular_exponentiation_to_computerB
                    (initial_state_circuit, L, a, modulus)
131
132         # Apply Inverse QFT to the second half of the
                    qubits
133         qft_dagger = QFT(num_qubits=L//2, inverse=True)
134         initial_state_circuit.append(qft_dagger, range(
                    L//2, L))
135
136         # Measure the second half of the register
137         initial_state_circuit.measure(range(L//2, L),
                    range(L//2, L))
138
139         # Measure the time for the second segment
140         end_time = time.time()
141         second_segment_time = end_time - mid_time
142         print("Time for second segment:",
                    second_segment_time, "seconds")

```

```

141
142     # Transpile the circuit
143     transpiled_circuit = transpile(
144         initial_state_circuit, backend=backend)
145
146     # Simulate the transpiled circuit
147     job = backend.run(transpiled_circuit)
148     result = job.result()
149     counts = result.get_counts()
150
151     # Extract measurement outcome
152     measurement_outcomes = list(counts.keys())
153     m_decimal = int(measurement_outcomes[0], 2)
154     m = m_decimal / (2**(t))
155
156     # Apply continued fractions algorithm to obtain
157     r
158     fractions = continued_fractions_algorithm(m)
159     r = best_r_from_continued_fractions(fractions)
160     print("Fractions:", fractions)
161     print("Estimated r:", r)
162
163     # Post-process to find factors
164     factors = post_process_factors(r, a, modulus)
165     print("Factors:", factors)
166
167     if factors is not None:
168         factors_found = True
169         print("Factors found:", factors)
170     else:
171         print("Factors not found, trying again...")
172
173     total_time = end_time - start_time
174     print("Total Elapsed Time:", total_time, "seconds")
175
176     if not factors_found:
177         print("Max iterations reached. Factors not
178         found.")
179
180 # Example usage
181 L = 8
182 a = 2 # Choose the base 'a' of the modular
183     exponentiation
184 modulus = 6059 # The modulus for the modular
185     exponentiation
186 run_shor_algorithm(L, a, modulus)

```

Listing 7: Order Finding Algorithm Function

Similarly to the standard version of **Shor's Algorithm**, Table 2 presents the results for several values of N.

Table 2: Results of Distributed Shor's Algorithm for several values of N

N	Average No. of Iterations	Factors Found	Average Time	Deviation
15	2.2	(5,3)	0.361 sec	0.365 sec
21	3	(3,7)	0.559 sec	0.498 sec
35	2.5	(5,7)	1.096 sec	1.197 sec
143	3.7	(11,13)	0.762 sec	0.295 sec
247	2.3	(19,13)	0.452 sec	0.387 sec
713	1.3	(31,23)	0.131 sec	0.252 sec
1927	2.8	(47,41)	0.561 sec	0.270 sec
3127	2.1	(59,53)	0.399 sec	0.331 sec
4757	1.6	(67,71)	0.204 sec	0.303 sec
6059	2.1	(73,93)	0.344 sec	0.343 sec

number 35, the following code was used to create a table of the simulation times.

```

1 import matplotlib.pyplot as plt
2
3 # List of times
4 times = [0.32637596130371094, 1.0828676223754883,
5 0.921400785446167, 0.004154205322265625,
6 0.5793805122375488, 0.0042269229888916016,
7 0.8766880035400391, 0.7545344829559326,
8 0.7607152462005615, 0.0060749053955078125]
9
10 # Calculate the average time
11 average_time = sum(times) / len(times)
12
13 # Create the histogram with enhanced styling
14 plt.figure(figsize=(10, 6))
15 plt.hist(times, bins=10, edgecolor='black', color='
16     skyblue', alpha=0.7)
17 plt.axvline(average_time, color='red', linestyle='
18     dashed', linewidth=2, label=f'Average: {
19     average_time:.2f}s')
20 plt.title('Distribution of Times', fontsize=16,
21     fontweight='bold')
22 plt.xlabel('Time (seconds)', fontsize=14)
23 plt.ylabel('Frequency', fontsize=14)
24 plt.grid(True, linestyle='--', alpha=0.7)
25 plt.xticks(fontsize=12)
26 plt.yticks(fontsize=12)
27 plt.legend(fontsize=12)

```

To represent the distribution of the time it takes to factor the


```

25 # Show the plot
26 plt.show()

```

Listing 8: Distribution of Total Time of Simulation for N

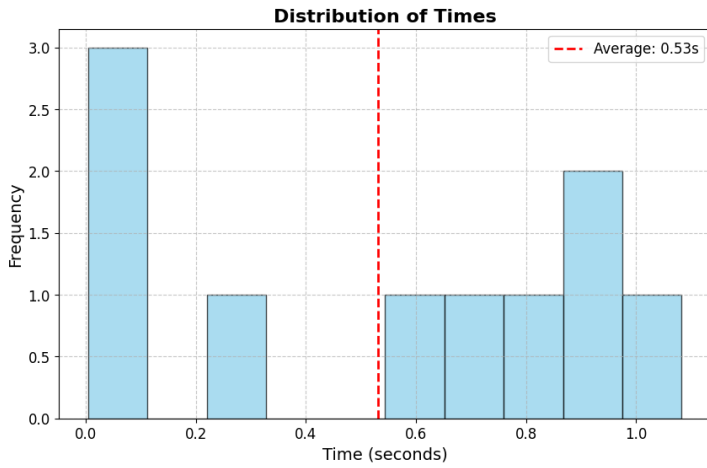


Fig. 8: Time Distribution of 10 Simulations N=35

The time for each computer was also measured and plotted for the same 10 tries, similarly to Figure 8.

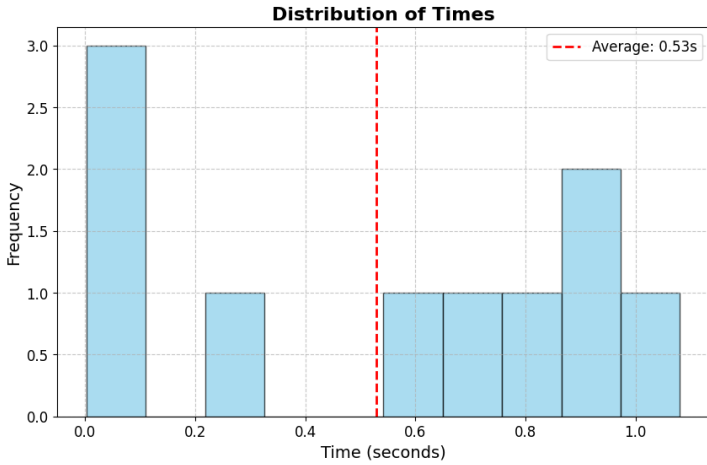


Fig. 9: Time Distribution of 10 Simulations N=35 for Computer A

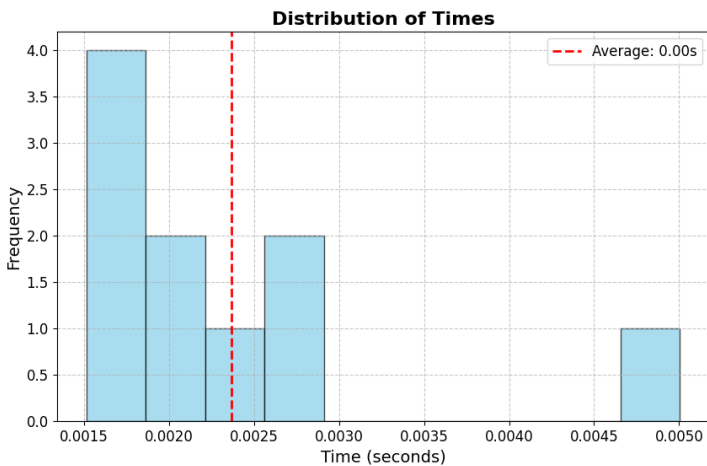


Fig. 10: Time Distribution of 10 Simulations N=35 for Computer B

11 Complexity Analysis [4]

To analyse the complexity of this algorithm four different parameters will be studied:

- **Space Complexity**
- **Time Complexity**
- **Circuit Depth**
- **Communication Complexity**

Before starting to analyse the different complexity parameters of the algorithm, through this section of the project we will assume that the **Modular Exponentiation** has a time complexity $\mathcal{O}(L^3)$.

11.1 Space Complexity

To analyse the **Space Complexity** it is necessary to analyse the number of qubits used during the circuit. To simulate **Shor's Algorithm** for computer A it is necessary to have $\frac{5L}{2} + 1 + \lceil \log_2(2 + \frac{1}{\epsilon}) \rceil + b$ and for computer B $\frac{5L}{2} + 2 + \lceil \log_2(2 + \frac{1}{\epsilon}) \rceil + b$.

The non distributed version of the **Order Finding Algorithm** requires $3L + 1 + \lceil \log_2(2 + \frac{1}{\epsilon}) \rceil + b$, so the distributed version reduces the amount of needed qubits by almost $\frac{L}{2}$ in each computer.

11.2 Time Complexity

The **Time Complexity** (can also be referred as **Gate Complexity**) has a complexity of $\mathcal{O}(L^3)$, since $C_t(M_a)$ can be implemented with complexity of $\mathcal{O}(tL^2)$.

11.3 Circuit Complexity

Even though both the distributed version and the non-distributed version present a **Circuit Complexity** of $\mathcal{O}(L^3)$, the distributed version will present lower complexity, since in reality the complexity is $\mathcal{O}(tL^2)$, and in the distributed version has a lower value of t .

11.4 Communication Complexity

In the version of the **Distributed Order Finding Algorithm** implemented here, there are L qubits that need to be teleported, so the complexity is of $\mathcal{O}(L)$.

12 Applications of Shor's Algorithm [9]

Shor's Algorithm is a very powerful tool in breaking public key systems, such as:

- **RSA Scheme**
- **Finite Filed Diffie-Hellman Scheme**
- **Elliptic Curve Diffie-Hellman key exchange**

The **RSA Scheme** is based on the assumption that factoring very large integers is not a feasible task. While that is true for classic

computers, that is not the case for quantum computers and quantum algorithms.

The **RSA Scheme** is composed of three stages, the creation of the keys, the encryption and the decryption. Let us assume that we have Alice and Bob trying to communicate.

Alice, the receptor of the message, will create a Public Key that will be sent Bob, the sender of the message, and a Private Key, that only Alice will have. This Public Key will be the product of two primes (p and q) and the Private Key will be the modular inverse of $e \pmod{(p-1)(q-1)}$, where $\text{GCD}(e, (p-1)(q-1)) = 1$.

Since the whole encryption process is based on the module of $(p-1)(q-1)$, by applying Shor's Algorithm to the Public Key, it will be possible to decrypt any message sent with the **RSA Scheme**.

13 Non-Distributed vs Distributed

The advantages of the **Distributed Version of Shor's Algorithm** are based on the **Complexity Analysis** done earlier. The most important advantages it presents are the lower **Space Complexity** and **Circuit Depth**.

The **Distributed Shor's Algorithm** requires each computer to have fewer qubits available to perform the operations. By requiring fewer qubits to perform each part of the algorithm, it is easier to obtain quantum computers that are both practical and feasible with current technology. This brings several advantages, such as, improved error management, simplified control and calibration of the system and less problems, like quantum entanglement.

14 Conclusion

The implementation of the distributed version of **Shor's algorithm** is very important in the realm of quantum computing, demonstrating the practicality and efficiency of performing quantum computations in a distributed manner. This project has demonstrated the advantages of decomposing a complex quantum algorithm into smaller, manageable tasks that can be executed across multiple quantum processors.

By breaking down the algorithm into smaller components, we were able to run these tasks simultaneously more efficiently. By analyzing the complexity of the **Non-Distributed** and **Distributed** version of **Shor's Algorithm** we demonstrated the advantages of the Distributed version.

In conclusion, this project has demonstrated the practical viability of distributed quantum computing and the implications it has for classical cryptography.

References

[1] M. A. Nielsen and I. L. Chuang, "Quantum computation and quantum information," in Quantum Computation and Quantum

Information, 2000.

- [2] A. J., A. Adedoyin, and J. A. et al, "Quantum algorithm implementations for beginners."
- [3] L. K. Grover, "A fast quantum mechanical algorithm for database search," pp. 212–219, 1996.
- [4] L. Xiao, D. Qiu, L. Luo, and P. Mateus, "Distributed Shor's algorithm," Quantum Information and Computation, Vol. 23, No. 12 (2023) 0027–0044, 2022.
- [5] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in Proceedings of the 35th Annual Symposium on Foundations of Computer Science. Santa Fe, NM, USA: IEEE Computer Society Press, 1994, pp. 124–134.
- [6] P. S. J and P. K. N, "Implementation of shor's algorithm in qiskit by quantum computing using ibmq," International Journal of Advanced Trends in Computer Science and Engineering, vol. 10, no. 2, 2021.
- [7] R. L. S. Jr, "Shor's factoring algorithm and modular exponentiation operators," 2023.
- [8] M. A. Nielsen, E. Knill, and R. Laflamme, "Complete quantum teleportation using nuclear magnetic resonance," Nature, vol. 396, pp. 52–55, 1998.
- [9] J. F. Costa and P. Gouveia, Matemática Discreta. IST Press, 2019, vol. 62.